

Quality Assurance for Generative AI

A systematic playbook for diagnosing, debugging, and preventing hallucinations and errors in LLM outputs.



WARNING



ERROR



SYSTEM OPTIMAL

```
function check_consistency(input_data):  
    if (hallucination_score > threshold):  
        return "WARNING: HALLUCINATION"  
    else:  
        return "VERIFIED"  
// END OF DIAGNOSTIC MODULE
```

```
function check_consistency(input_data):  
    if (hallucination_score > threshold):  
        return "WARNING: HALLUCINATION"  
    else:  
        return "VERIFIED"  
// END OF DIAGNOSTIC MODULE
```

```
function check_consistency(input_data):  
    if (hallucination_score > threshold):  
        return "WARNING: HALLUCINATION"  
// END OF DIAGNOSTIC MODULE
```

```
function check_consistency(input_data):  
    if (hallucination_score > threshold):  
        return "WARNING: HALLUCINATION"  
    else:  
        return "VERIFIED"  
// END OF DIAGNOSTIC MODULE
```

```
function check_consistency(input_data):  
    if (hallucination_score > threshold):  
        return "WARNING: HALLUCINATION"
```

```
        return "VERIFIED"  
// END OF DIAGNOSTIC MODULE
```

```
function check_consistency(input_data):  
    if (hallucination_score > threshold):  
        return "WARNING: HALLUCINATION"  
    else:  
        return "VERIFIED"  
// END OF DIAGNOSTIC MODULE
```

```
function check_consistency(input_data):  
    if (hallucination_score > threshold):  
        return "WARNING: HALLUCINATION"  
    else:  
        return "VERIFIED"  
// END OF DIAGNOSTIC MODULE
```

```
function check_consistency(input_data):  
    if (hallucination_score > threshold):  
        return "WARNING: HALLUCINATION"  
    else:  
        return "VERIFIED"  
// END OF DIAGNOSTIC MODULE
```

```
function check_consistency(input_data):  
    if (hallucination_score > threshold):  
        return "WARNING: HALLUCINATION"  
// END OF DIAGNOSTIC MODULE
```

```
function check_consistency(input_data):  
    if (hallucination_score > threshold):  
        return "WARNING: HALLUCINATION"  
    else:  
        return "VERIFIED"  
// END OF DIAGNOSTIC MODULE
```

```
function check_consistency(input_data):  
    if (hallucination_score > threshold):  
        return "WARNING: HALLUCINATION"  
    else:  
        return "VERIFIED"  
// END OF DIAGNOSTIC MODULE
```

```
function check_consistency(input_data):  
    if (hallucination_score > threshold):  
        return "WARNING: HALLUCINATION"  
    else:  
        return "VERIFIED"  
// END OF DIAGNOSTIC MODULE
```

[DIAGNOSTIC PHASE: THE STAKES]

 [WARNING] Customer Service Bots	Risk Profile: <u>High public exposure.</u>	Failure Impact: Brand damage and customer mistrust from confidently delivered misinformation.
 [ERROR] Data Analysis	Risk Profile: <u>Operational integrity.</u>	Failure Impact: Cascading operational failures due to reproducible math errors or faulty logic in data interpretation.
 [CRITICAL FAULT] Decision Support Systems	Risk Profile: Strategic execution.	Failure Impact: Costly leadership missteps driven by fabricated facts or fake citations disguised as historical data.

Key Takeaway: Misunderstanding an AI's mistake transforms a minor technical glitch into a costly business outcome.

[WARNING: FABRICATION DETECTED]

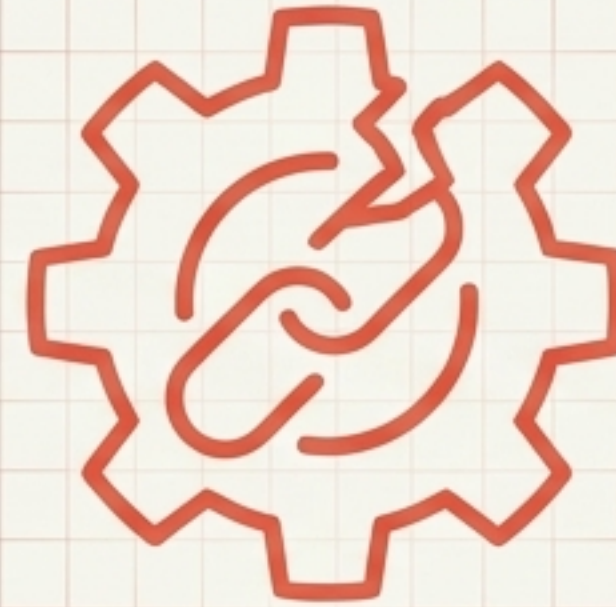
The Hallucination



The AI invents information. It fabricates facts, sources, or events that do not exist, presenting them with absolute certainty.

[FATAL: LOGIC FAULT]

The Error



The AI applies faulty logic to real data. It fails at reasoning, relies on outdated knowledge, or executes poor math.

Knowing the difference dictates the mitigation strategy.

Hallucinations vs. Errors: The Diagnostic Matrix

Dimension	 The Hallucination	 The Error
Definition	Fabrication of facts, entities, or sources	Faulty logic or misapplied rules
Root Cause	Probabilistic next-word guessing prioritizing fluency over truth	Outdated training data, broken dependencies, or bad math
Key Identifiers	Fake citations, confident falsehoods, unverifiable claims	Reproducible mistakes, syntax syntax breaks, flawed reasoning chains
Resolution Strategy	Fact-checking against external ground truth	Debugging, logic testing, and parameter adjustment

Anatomy of a Confident Falsehood

[PROMPT]

Summarize the 2024 ruling in Smith v. CyberTech.

[AI OUTPUT]

In the landmark 2024 case Smith v. CyberTech, the Supreme Court ruled 7-2 that...

Fabricated Entity.

The case does not exist in any legal database.

Confident Falsehood.

The model pairs 'Supreme Court' with standard ruling ratios to simulate authority.

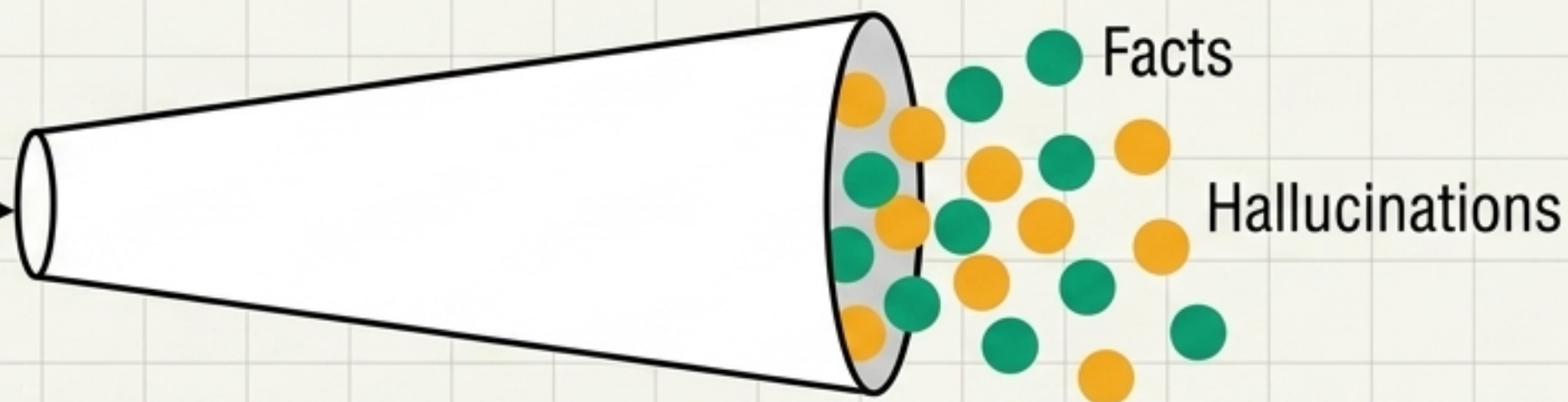
Fake Citation. URL structure mimics real journals but leads to a dead link.

Fluency does not equal accuracy. AI is designed to sound plausible, even when inventing reality.

The Prompt-to-Output Funnel

The Unstructured Prompt

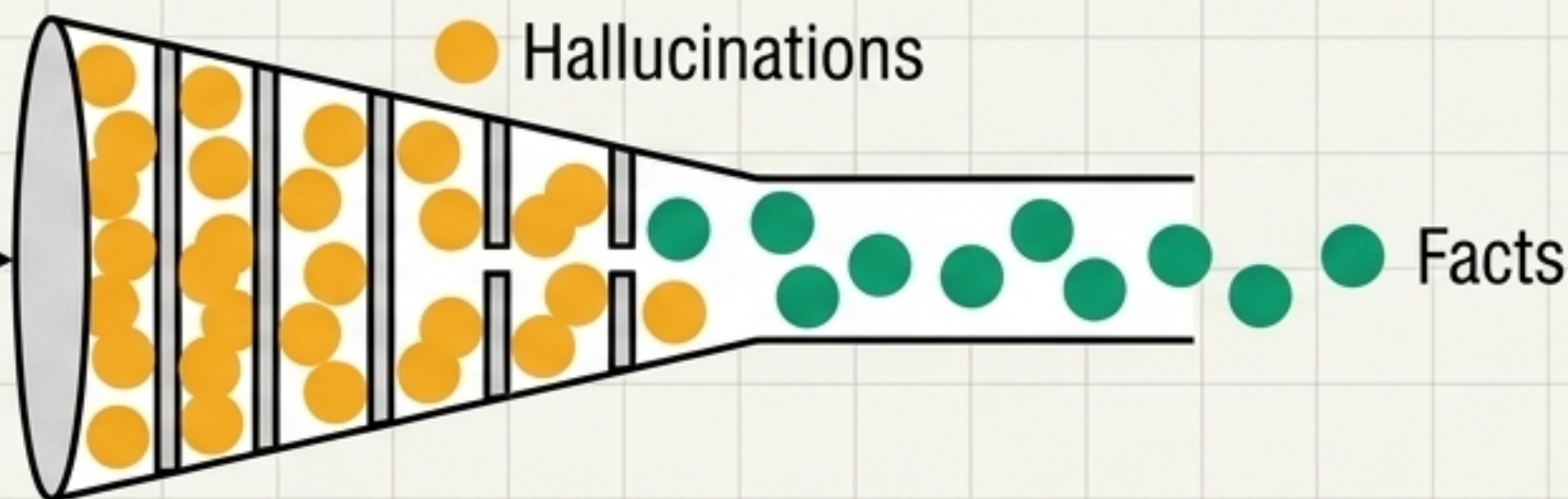
Input: Write a report on market trends.



High risk of confident falsehoods.

The Constrained Prompt

Input: Using only the provided Q3 CSV data, summarize market trends. Do not include outside knowledge. Cite the exact row for each claim.



Constraining the operational boundaries drastically reduces hallucination risk.

Resolving Hallucinations: Fact-Checking Protocols

A Quick Reality-Check Pipeline



Step 1

Isolate Factual Claims

Strip away the narrative wrapper. Extract only names, dates, statistics, and citations for independent review.



Step 2

Verify Ground Truth

Cross-reference the isolated claims against primary sources or trusted internal databases. Never use an LLM to verify another LLM without a web-browsing grounding tool.

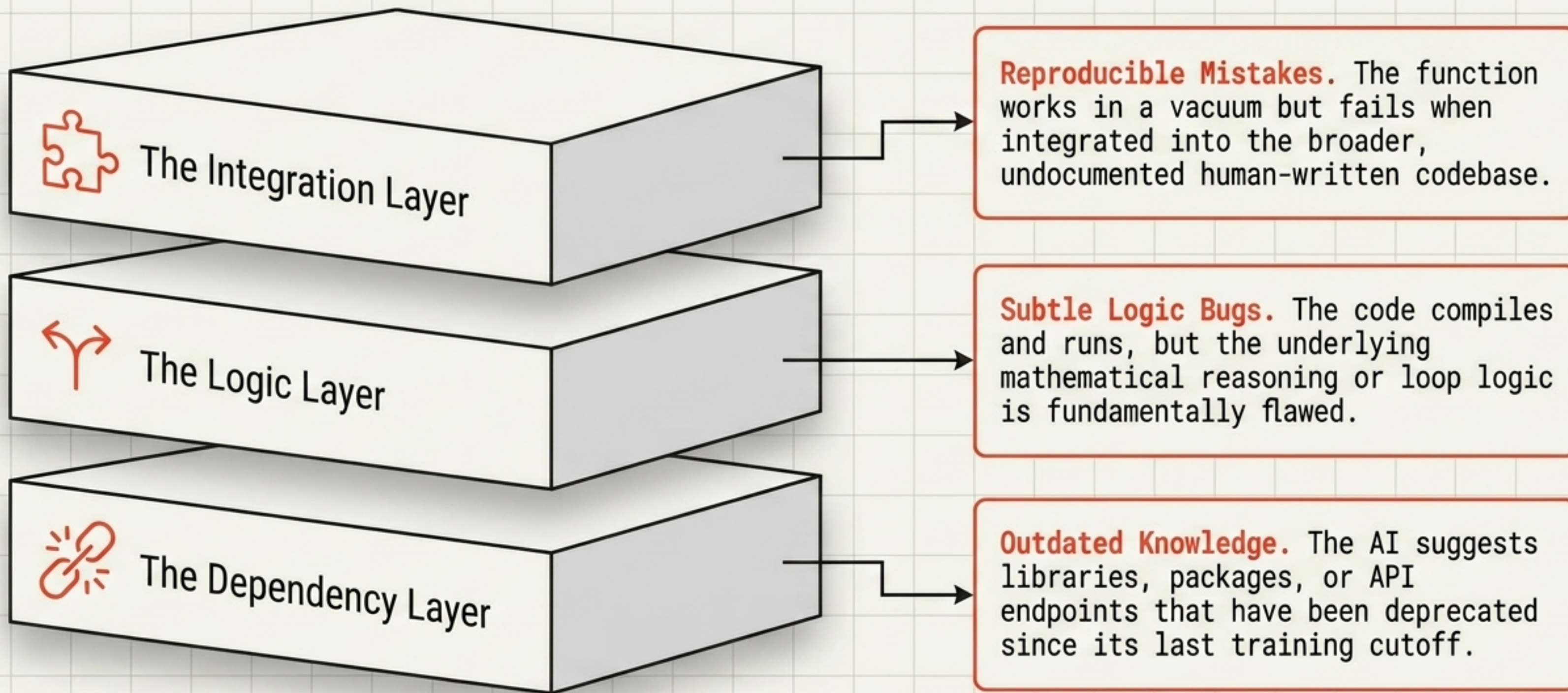


Step 3

Invert the Query

Ask the AI to argue the opposite or highlight weaknesses in its own output. This breaks the probabilistic "yes-man" loop and forces logical evaluation.

The Vulnerability Surfaces of AI Code Assistants



Code Assistant Debugging Workflow

[UNVERIFIED AI GENERATION]

```
1 def process_data(arr):  
2     return arr.sort()
```

Subtle logic bug: Fails on edge cases. Returns None instead of the sorted list.

Unit Testing &
Code Review

[VERIFIED & DEPLOYABLE]

```
1 def process_data(arr):  
2     return sorted(arr)
```

Logic corrected via isolated unit tests. Output is safe and deterministic.

The Human-AI Verification Interface

1. AI Generation (Speed)

The LLM rapidly produces draft text, data interpretations, or boilerplate code.



2. Automated Check (Scale)

Linters, unit tests, and structural validation scripts catch reproducible syntax errors and surface-level faults.



3. Human Review (Nuance)

The tech professional manually evaluates the output for subtle logic bugs, contextual accuracy, and strategic alignment.



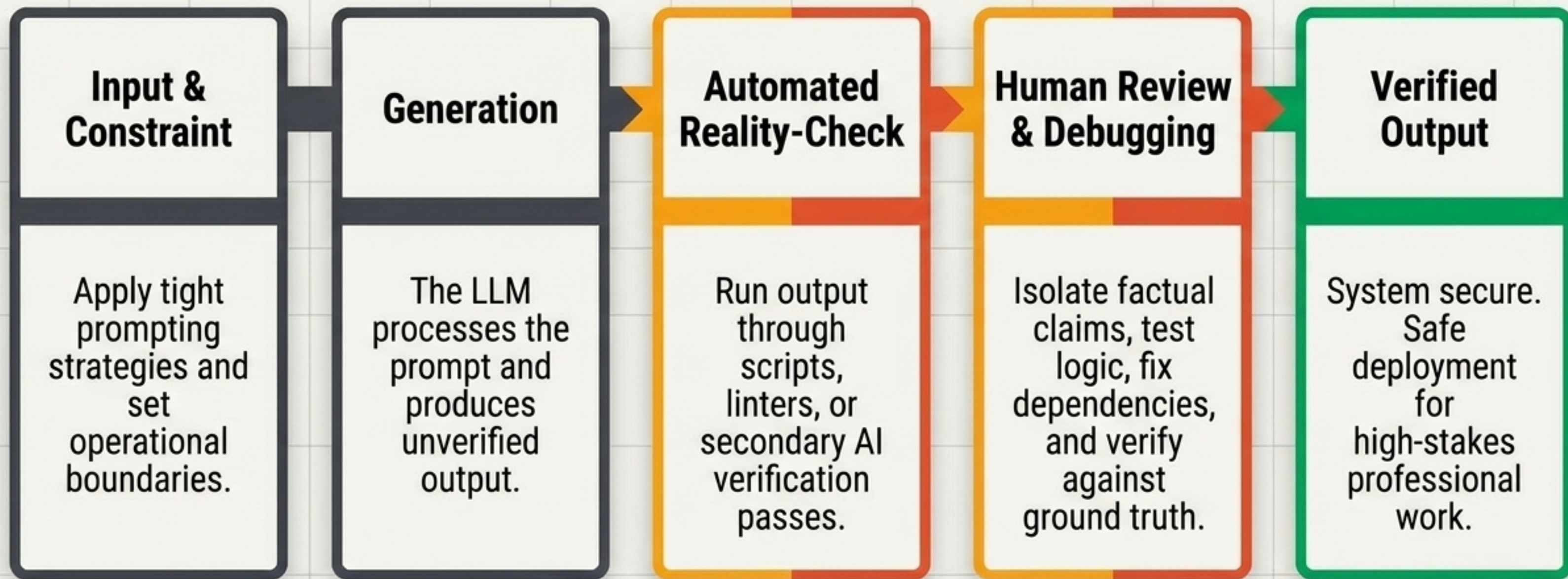
4. Final Solution (Reliability)

The combined, scrubbed output is deployed to production.



Reliable output is never generated; it is engineered through combined human and automated scrutiny.

The Universal Verification Framework



A Model-Agnostic Methodology



The architectural vulnerabilities of LLMs—probabilistic guessing and static training data—are inherent to the technology, not the brand. This systematic framework ensures quality and safety regardless of the underlying model.



Trust Requires Verification

Generative AI operates at the intersection of remarkable fluency and absolute **probabilistic uncertainty**. By replacing blind trust with a rigorous, diagnostic workflow—separating hallucinations from mechanical errors—you transform AI from a liability into a reliable engine for professional productivity.

[END OF PLAYBOOK // SYSTEM SECURE]